

MyScoutee Patterns, Data Structures and Algorithms

Application / laboratories

An illustrated map of the implemented design patterns, data structures and algorithms in the MyScoutee application and its adjacent laboratories.

Contents

Implemented patterns, data structures and algorithms

Laboratory and supporting-system map

Implemented patterns, data structures and algorithms

This guide maps concrete implementations in the application and adjacent laboratories. Pattern names describe the code; they are not claims that every class or library operation is listed.

Pattern map by ownership boundary

Pattern	Implemented example	Why the boundary exists
Domain-oriented modular monolith	`server/projects/profile/src/main/java/.../domain/` groups accounts, events, affinity, payments and other product areas	One canonical Java source owner keeps domain behavior together while allowing alternate assemblies
Ports and adapters	Domain `api` interfaces, repositories, HTTP controllers and `transport` remote implementations	Business contracts are separated from storage and cross-process delivery
Adapter selection	Angular `shared/core/base/adapter-loaders.ts`, `local-adapters.ts`, `http-entry-adapters.ts`	The same UI can use browser-local examples or server HTTP; local behavior is a distinct evidence boundary
Repository and DTO mapping	Java domain repositories/mappers and Angular converters	Persistence and wire/UI representations are translated at explicit edges
Reactive store and presentation projection	Angular `shared/ui/context/stores/` and converters	UI state, remote loading and rendered rows are kept in their owning feature slices
Durable work queue	`AffinityRecomputeTaskService`, worker and Mongo change-stream wake-up	A domain mutation records work; retries and delayed availability survive process restarts
Transactional outbox-style delivery	Email and notification outbox documents/workers; Registry reporting has its own durable path	External delivery is decoupled from the user request and can be retried with recorded state
Gateway and contract-bound remote adapter	nginx public route map, MCP gateway, OpenAPI transport contracts and generated service bindings	Public routes, personal MCP operations and internal service calls have separate permission surfaces
Configuration activation	`OperatorConfigurationService` and `DeploymentFirebaseRuntime`	Staged provider settings are tested and explicitly activated before runtime use
Idempotent external operation	Payment provider keys and webhook processing; simulator replay exercises the contract	Repeated callbacks or retries must not duplicate the business transition

The KVM assembly is a deployment arrangement of the Java domains. A remote implementation of an `api` interface is selected by transport binding; it does not make every Java package an independently deployed service. The shared `myscoutee-base`, `myscoutee-components`, `myscoutee-mcp` and `myscoutee-microservices` repositories supply reusable libraries/runtime. Product rules and configuration remain application-owned.

Data structure map

Structure	Concrete use	Invariant or limit
Mongo document and indexed collection	Users, events, affinity tasks, outboxes and projections	Atomicity is at document scope; a cross-store restore must reconcile Mongo with objects
Graph nodes and weighted edges	`events/algorithm/dto/{Graph,FGraph,Node,Edge}` and affinity graph views	Identity is a node ID; filters remove ineligible or disabled users before planning
Adjacency maps and sets	`StableGroupPlanner.GraphSnapshot`, admin graph services	Fast neighbor lookup and deduplication support connected-component and score operations
`PriorityQueue<Edge>`	`events/algorithm/dto/CNode`	Orders candidate edges by its comparator; tie and order rules belong to the implementation
`HashMap`/`HashSet` and `LinkedHashMap`/`LinkedHashSet`	Group planning and graph snapshots	Hash collections support membership; linked collections preserve iteration order where output stability matters
Deque and list	Graph traversal/group-building and ordered candidate/output sequences	A deque represents traversal frontier; list order is meaningful for invitations and pagination
Durable task and outbox state	Mongo task/outbox documents with status, key and availability time	A queued state is persisted, not merely an in-memory event
Browser IndexedDB and store state	Angular local adapters, caches and outboxes	Browser data cannot stand in for server authorization or persistence proof
SQLite state in adjacent services	Registry, 2FA and payment simulator	Each belongs to its own service/lab; these databases are not product Mongo replicas

Algorithm map

Algorithm or computation	Implementation owner	Result and evidence boundary
Stable group and invitation planning	`events/algorithm/StableGroupPlanner`, `Algo`, `EventGeneratorByPriority`	Filters eligible graph nodes, finds connected components, builds size/type-constrained groups, refines them with swaps and selects invite batches. Unit/E2E tests cover selected behavior; it is not a guarantee of a global optimum.
Alternative event generation	`EventGeneratorByRandom`, `EventGeneratorByScore`, `EventGeneratorByMingle`	Explicit generators select different candidate strategies. Their requests and results use `GeneratorModels`; the selected strategy is a product decision.
Affinity recomputation	Affinity task/change-stream/worker services	Coalesces keyed work, claims due tasks and refreshes projections after relevant changes; reconnect scans and timers cover wake-up gaps and future work.
Graph statistics	`AdminGraphScienceService`	Computes degree, connected/community measures, local clustering, bridge-like signals and aggregate health scores from product graph data. These are application-defined indicators, not scientific validation of real-world social outcomes.
Graph forest and layout projection	`AdminAffinityGraphService` and layout worker	Builds filtered components/forests and materialized layout pages/tiles for administration; layout is a derived view, not canonical user data.
Counters and projections	Group, campaign and community counter workers	Recomputes or propagates derived counts from domain records; a displayed count is a projection with its own refresh path.
Token and identity verification	Firebase verifier, account session filter and integration-token policy	Checks credentials and route/domain permissions before business access; a provider token alone does not grant an application role.

Reading and change rule

For a specific algorithm, inspect its current source, DTOs, tests and call sites before changing its behavior. This catalog intentionally does not claim asymptotic complexity, optimality, fairness or production performance where no matched measurement establishes them. The source repository's tests and runtime evidence remain authoritative.

Algorithms and structures in the separate laboratories

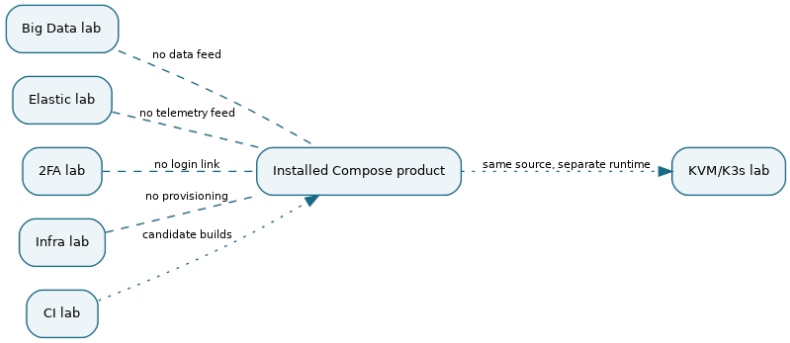
Laboratory	Pattern or computation	Structures and evidence limit
Big Data	Event-stream replay, optional schema evolution, columnar materialized views, batch aggregation and stateful stream rollups	Kafka topics/partitions/offsets, Avro schema IDs, ClickHouse tables/views, Spark DataFrames/Parquet partitions, Flink checkpoints/windows. Inputs are synthetic; no live product affinity feed.
Elastic / CodeForge	Deterministic source selection, indexed search, highlighted relevance results, heuristic findings and detection rules	Elasticsearch documents and inverted indexes; Kibana views/dashboards; Beats streams. Finding rules are work markers, not confirmed defects; telemetry is synthetic or local.
2FA	TOTP time-step verification, HOTP counter window, one-use recovery codes and WebAuthn challenge/response	SQLite factor/ceremony state; hashed recovery codes; public-key credentials/counters. The browser demo and optional Keycloak identity trial do not issue product sessions.
Security	Isolated source/image/passive-web scan pipeline and human finding review	Tool reports and scan artifacts. A finding is not an exploit proof or certification.
Infra	Declarative convergence and repeat-run comparison	Ansible tasks and Puppet resources target the same disposable nginx VM; a clean repeat run demonstrates only that lab's desired state.
CI and SonarQube	Dependency-aware build gates, retained candidate artifacts and static/source analysis	GitLab jobs/artifacts and Sonar analysis snapshots, using native test/coverage reports. Neither proves deployed behavior without runtime QA.
MCP adapter	Contract catalog intersection with live authenticated backend capabilities	Product/contract JSON and per-connection OAuth context; configured tools do not grant permission.
Payment simulator	Idempotency key, signed callback validation and replay of the same event	SQLite sessions/events/audit records and HMAC signatures; test provider semantics, no real card processing.
Registry	Signed deployment identity and durable reporting/receipt protocol	Ed25519 identity plus SQLite protocol state/receipts; Explore and external Registry have separate keys and data.

This table maps the named mechanisms at their owning boundary. Detailed mathematical parameters, provider protocol versions, database schemas and performance measurements remain in each laboratory's source and tests.

Laboratory and supporting-system map

A laboratory capability is not automatically part of the installed product. The sections below show what each lab does and where its data stays.

Runtime relationship



The installed product, backend source Compose forms and KVM lab have different deployment and qualification evidence. Big Data, Elastic, 2FA, security and infrastructure labs are standalone. CI and SonarQube analyze/build sources; they are engineering systems, not member-facing product services.

Capability and data boundaries

Backend source Compose

What runs or is evaluated: Development, E2E and production source-stack models
Input and state: REAL/DEMO Mongo, Redis, objects, local adapters/simulators by profile
Product relationship: Canonical product source, but source stacks are not the installed release

`myscoutee-microservices`

What runs or is evaluated: KVM/K3s, Helm, eight domain assemblies, gateway and monitoring
Input and state: Same application Java source; lab Mongo replica set and PVCs
Product relationship: Separate deployment experiment; no automatic release qualification

`myscoutee-bigdata`

What runs or is evaluated: Kafka/KRaft, optional Avro Schema Registry, ClickHouse, Spark/Parquet/SeaweedFS, Flink
Input and state: Synthetic event stream, analytical tables and local lake
Product relationship: Does not consume product Mongo/Redis or replace affinity worker

`myscoutee-elastic`

What runs or is evaluated: CodeForge source search, Elasticsearch/Kibana and Beats; separate optional Fleet
Input and state: Read-only source mount, local index and synthetic MyScoutee telemetry
Product relationship: Local evaluation, not product search, SIEM or production telemetry

`myscoutee-2fa`

What runs or is evaluated: Go TOTP/HOTP, WebAuthn, recovery codes; optional Keycloak OIDC/SAML trial
Input and state: Its own SQLite and encryption key
Product relationship: Standalone factor API; no current MyScoutee login connection

`myscoutee-security`

What runs or is evaluated: Source, image and passive web assessment workflows
Input and state: Isolated scan artifacts
Product relationship: Findings need review; scans neither mutate product data nor certify it

`myscoutee-infra`

What runs or is evaluated: Vagrant VM with Ansible/Puppet comparison
Input and state: Same synthetic nginx health page, disposable guest
Product relationship: Does not provision product or production nginx

`myscoutee-ci`

What runs or is evaluated: Local GitLab, runners, registry and manual release gates
Input and state: Source checkouts, job outputs and retained candidates
Product relationship: Delivery laboratory; application packages/images still have their owners

`myscoutee-sonarqube`

What runs or is evaluated: One SonarQube/PostgreSQL instance and selected source analyses
Input and state: Native test/coverage reports and scan snapshots
Product relationship: Quality evidence, not runtime traffic or a product store

Adjacent implementation and test owners

Owner	Scope	Boundary
`myscoutee-mcp`	Shared Go MCP HTTP/stdio adapter and live backend capability filtering	Application-owned `product.json` and `contract.json`; backend owns identity and actions
`myscoutee-payment-simulator`	Stripe/Barion-shaped delayed capture, signed callbacks, replay and SQLite audit	Development/QA only; no card data and absent from production topology
`myscoutee-registry`	Independent Go/SQLite network directory and signed protocol	External to product; opt-in enrollment and separate Explore instance
`myscoutee-nginx`	Reusable edge image/configuration	Application route map and deployment settings select public surfaces
`myscoutee-packaging`	Shared Debian/package mechanisms	Backend/template own product inputs, images, migrations and release qualification
`myscoutee-e2e` and `myscoutee-qa`	Installation-level and manual QA evidence	Results describe the tested build, fixtures and runtime
`myscoutee-agent` and `myscoutee-skills`	Advisory trust prototype and agent instructions	Engineering workflow tools, not product AI services

The frontend and shared components/base repositories are application dependencies, not separate experiments. The template is another product built from shared foundations and retains its own behavior and deployment configuration. No adjacent repository should be shown as a live data-flow arrow unless its integration is present in the product configuration and verified end to end.